

P1	P2	P3	Total

Test laborator - Numărul II

*Pentru rezolvarea problemelor puteți defini predicate ajutătoare, puteți folosi predicate pre-definite, dar **nu** puteți folosi meta-predicate care colectează toate soluțiile (de tipul findall, findnsol, bagof, setof, etc.). Nu tratați cazuri de eroare în afara celor cerute explicit de problemă.*

(P1) [5 puncte]

Scrieți un predicat `aparitii(ListaCN, C1,C2)` astfel încât:

- `ListaN` este o listă cu termeni de forma `cn(cuvant, 5)`; în această listă un același cuvânt poate să apară de mai multe ori (vă puteți imagina că al doilea argument din `cn(cuvant, 2)` este numărul de apariții ale cuvântului într-un paragraf, iar textul are mai multe paragrafe.
- pentru `C1` și `C2` date, predicatul `aparitii(ListaCN,C1,C2)` este adevărat dacă suma numerelor `N1` din termenii `cn(C1,N1)` este egală cu suma numerelor `N2` din termenii `cn(C2,N2)` (cuvintele `C1` și `C2` apar de același număr de ori în întregul text).

Exemplu:

```
?- aparitii([cn(apa,4), cn(si,9), cn(apa,2), cn(pahar,6)], apa, pahar)
true
?- aparitii([cn(apa,4), cn(si,9), cn(apa,2), cn(pahar,6)], apa, si)
false
```

(P2) [10 puncte] Scrieți un predicat `exista(KBC, N,MultimeC)` astfel încât:

- `KBC` este o listă care conține termeni de forma `c(ada, 3)` (primul argument este un cuvânt, iar al doilea argument este lungimea cuvântului respectiv).
- pentru un număr natural `N` dat, `MultimeC` să fie instanțiată cu listă de cuvinte din `KBC` cu proprietatea că suma lungimilor lor este egală cu `N`.

Precizări: în `KBC` fiecare cuvânt apare o singură dată; `MultimeC` trebuie să fie o mulțime (fiecare element apare o singură dată și ordinea elementelor nu contează).

Exemplu:

```
exista([c(testul,6), c(usor,4),c(este,4),c(ada,3),c(carte,5)],9,MultimeC).
MultimeC=[ada,testul].
```

(P3) [15 puncte]

În acest exercițiu vi se cere să evaluați o expresie într-o stare de memorie dată. Variabilele care apar în expresii sunt definite cu ajutorul predicatului `isvar`:

```
isvar(X) :- member(X,[u,v,w,x,y]).
```

Fiind dată o mulțime de variabile, și un atom `nul`, o expresie este un termen Prolog definit recursiv astfel: `nul` este expresie; orice variabilă este expresie; dacă `E1` și `E2` sunt expresii, atunci `ad(E1,E2)` și `mu(E1,E2)` sunt expresii (`ad` are semnificația "adunare", `mu` are semnificația "multiplicare").

Exemple de expresii:

```
u,v,ad(nul,x), mu(x,y), ad(mu(x,y),nul), mu(ad(nul,u),ad(v,nul))
```

- (a) [5 puncte] Scrieți un predicat `simplifica(E,ES)` astfel încât `E` este o expresie dată, iar `ES` este expresia în care sunt simplificate operațiile cu `nul`, după regulile:

- forma simplificată pentru `ad(X,nul)` și `ad(nul, X)` este `X`,
- forma simplificată pentru `mu(X,nul)` și `mu(nul, X)` este `nul`.

Exemplu:

```
simplifica(ad(nul,v),v).  
simplifica(mu(v,nul),nul).  
simplifica(mu(ad(nul,v),ad(u,nul)), mu(v,u)).
```

- (b) [10 puncte] Presupunem ca se dă o bază de cunoștințe care conține fapte de forma `val(v,5)`. (primul argument al predicatului `val` este o variabilă, iar al doilea argument este valoarea variabilei respective); de exemplu:

```
val(u,3). val(v,-4). val(x,2). val(y,7). val(w,7).
```

Scrieți un predicat `valoare(Expr, Rez)` astfel încât, dacă `Expr` este o expresie dată, `Rez` va fi valoarea expresiei în care valorile variabilelor sunt date de baza de cunoștințe, iar valoarea lui `nul` este 0.

Exemplu:

```
valoare(ad(mu(u,v),x), Rez).  
Rez = -10.
```

```
valoare(w,Rez).  
Rez=7.
```

Atenție: vom presupune că expresiile sunt corecte; la punctul (b) presupunem ca toate variabilele care apar în expresie apar și în baza de cunoștințe; variabilele apar în baza de cunoștințe o singură dată.